Java Day 11：面向对象进阶

多态（Polymorphism） · 向上转型 · 动态绑定

今日目标：理解多态成立条件；掌握向上转型；区分编译时类型与运行时类型；理解动态绑定；会使用 instanceof 做基本类型判断。

一、建议时间安排

环节	内容	时间
1	多态概念与成立条件	15 分钟
2	向上转型与方法调用规则	20 分钟
3	动态绑定与类型转换	15 分钟
4	性能测试场景实战	20 分钟
5	独立练习与验收	20 分钟

二、什么是多态?

多态可以理解为：同一个父类引用指向不同的子类对象，调用同一个被重写的实例方法时，可以表现出不同的行为。

基本成立条件

1. 存在继承关系。
2. 子类重写父类的实例方法。
3. 使用父类类型的引用指向子类对象。

```
ApiResult result =
        new LoginApiResult(900, 500, "Alice");

result.showResult();
```

变量引用类型是 ApiResult，实际创建的对象类型是 LoginApiResult。调用重写方法时，会执行实际对象对应的子类实现。

三、向上转型（Upcasting）

向上转型就是将子类对象赋值给父类类型的引用。语法：父类类型 变量名 = new 子类构造方法();

```
ApiResult result =
        new LoginApiResult(900, 500, "Alice");
```

编译时类型（引用类型）是 ApiResult；运行时类型（实际对象类型）是 LoginApiResult。

向上转型后能调用什么?

```
ApiResult result = new LoginApiResult("Alice");

result.showResult();  // Parent declares this method
result.isSuccess();   // Parent declares this method
// result.showUsername(); // Compile error
```

能调用哪些方法，主要由编译时类型决定。被重写的实例方法最终执行哪个版本，由运行时实际对象类型决定。子类独有的方法不能通过父类引用直接调用。

四、动态绑定（Dynamic Binding）

```
ApiResult result1 =
        new LoginApiResult(900, 500, "Alice");
ApiResult result2 =
        new QueryApiResult(200, 200, 20);

result1.showResult(); // LoginApiResult implementation
result2.showResult(); // QueryApiResult implementation
```

两个变量的引用类型都是 ApiResult，但实际对象不同。运行时根据实际对象选择对应的重写方法，这就是动态绑定。

编译时类型与运行时类型

对比	编译时类型	运行时类型
含义	变量声明时的类型	实际创建的对象类型
例子	ApiResult	LoginApiResult
主要影响	决定可调用哪些方法	决定重写方法执行哪个版本

注意：多态主要针对实例方法重写。字段访问和 static 方法不按照实例方法的动态绑定规则处理。

五、向下转型与 instanceof

有时需要访问子类独有的方法，可以向下转型，但必须确保实际对象确实是该子类。

```
ApiResult result = new LoginApiResult(900, 500, "Alice");

if (result instanceof LoginApiResult) {
    LoginApiResult loginResult =
            (LoginApiResult) result;
    loginResult.showUsername();
}
```

如果把一个实际的父类对象强制转换成不匹配的子类，运行时会抛出 ClassCastException。instanceof 可在转换前判断对象类型或其子类型关系。

六、综合实战：统一管理不同接口结果

父类 ApiResult.java

```
public class ApiResult {
    private int responseTime;
    private int statusCode;

    public ApiResult(int responseTime, int statusCode) {
        this.responseTime = responseTime;
        this.statusCode = statusCode;
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }

    public boolean isSlow() {
        return responseTime > 500;
    }

    public void showResult() {
        System.out.println("Response Time: " + responseTime + "ms");
        System.out.println("Status Code: " + statusCode);
        System.out.println("Success: " + isSuccess());
        System.out.println("Slow: " + isSlow());
    }
}
```

子类 LoginApiResult.java

```
public class LoginApiResult extends ApiResult {
    private String username;

    public LoginApiResult(int responseTime, int statusCode,
                          String username) {
        super(responseTime, statusCode);
        this.username = username;
    }

    @Override
    public void showResult() {
        System.out.println("Request Type: Login");
        System.out.println("Username: " + username);
        super.showResult();
    }
}
```

子类 QueryApiResult.java

```
public class QueryApiResult extends ApiResult {
    private int recordCount;

    public QueryApiResult(int responseTime, int statusCode,
                          int recordCount) {
        super(responseTime, statusCode);
        this.recordCount = recordCount;
    }

    @Override
    public void showResult() {
        System.out.println("Request Type: Query");
        System.out.println("Record Count: " + recordCount);
        super.showResult();
    }
}
```

测试类 Main.java

```
public class Main {
    public static void main(String[] args) {
        ApiResult[] results = {
            new LoginApiResult(900, 500, "Alice"),
            new QueryApiResult(200, 200, 20),
            new LoginApiResult(150, 200, "Bob"),
            new QueryApiResult(800, 200, 100)
        };

        for (ApiResult result : results) {
            System.out.println("--------------------");
            result.showResult();
        }
    }
}
```

重点：数组类型是 ApiResult[]，但可以保存不同子类对象。循环统一调用 result.showResult()，每个对象都会执行自己的重写版本，无需按类型写大量 if-else。

七、性能测试工程中的应用

性能分析程序可能处理登录、查询、下单、支付等不同接口结果。它们共享耗时、状态码、成功判断等通用信息，同时又有各自的业务字段。通过父类统一管理共性，再由子类扩展差异，能减少重复代码并提升扩展性。

八、今日作业

任务 1：创建父类 ApiResult，包含 private 属性 responseTime、statusCode；构造方法；isSuccess()；isSlow()；showResult()。

任务 2：创建 LoginApiResult 和 QueryApiResult 两个子类，分别增加 username 和 recordCount 属性，使用 super(...) 初始化父类，并重写 showResult()。

任务 3：在 Main 中创建 ApiResult[] 数组，放入至少 4 个不同子类对象，通过 for-each 循环统一调用 showResult()。

任务 4：思考题：

① 什么是向上转型？

② 父类引用为什么不能直接调用子类独有方法？

③ 编译时类型与运行时类型分别影响什么？

④ 为什么同一个父类数组可以保存不同子类对象？

⑤ 多态如何减少 if-else？

九、验收标准

- [ ] 两个子类均正确继承 ApiResult。
- [ ] 两个子类均正确重写 showResult()。
- [ ] ApiResult[] 能保存不同子类对象。
- [ ] for-each 循环能触发各自子类的重写方法。
- [ ] 能解释向上转型、动态绑定和编译时/运行时类型。
- [ ] 能解释向下转型及 instanceof 的基本用途。

提交方式

完成后上传 ApiResult.java、LoginApiResult.java、QueryApiResult.java、Main.java，附上控制台输出和 5 道思考题答案。我会逐个检查，并尽量实际编译运行后再批改。

下一课：Day 12 将继续学习抽象类与接口，理解如何定义统一规范。